GUIA VISUAL 058

LAÇOS
FOR · WHILE · UNTIL

*O expurgo da madrugada rodava havia meses até a coleta gravar `relatorio de marco.log`. Naquela noite o `gzip` foi chamado três vezes — com relatorio, de e marco.log —, o log real continuou crescendo e um mês depois a partição encheu. O erro não estava no `for` nem no `gzip`: estava na **fonte da lista**.*

058
GUIAS LINUX

DE ONDE VEM A LISTA — O LAÇO ITERA PALAVRAS, NÃO ARQUIVOS

`for f in $(ls *.log)`	`relatorio` `de` `marco.log`	**3 iterações.** A saída virou **texto** e o `IFS` recortou nos espaços; um nome com `*` ainda se expandiria outra vez. Não existe caso em que iterar sobre `ls` seja melhor.
`for f in *.log`	`relatorio de marco.log`	**1 iteração.** O *globbing* entrega **itens**, não uma cadeia de caracteres: espaço e quebra de linha sobrevivem inteiros. Vale igual para `{1..10}` e `"${arr[@]}"`.
`while IFS= read -r -d '' f` `  < <(find … -print0)`	`relatorio de marco.log`	**1 iteração**, e também em subdiretório e com `\n` no nome. O separador é o **byte nulo** — o único caractere que o kernel proíbe num nome de arquivo.

Glob que não casa não vira lista vazia: em diretório sem `*.log`, o laço roda **uma vez** com `f` valendo o texto literal `*.log`. Conserte com `shopt -s nullglob` ou com `[[ -e $f ]] || continue` na primeira linha do corpo. E se o corpo é uma chamada única a um comando, dispense o laço: `find … -exec gzip {} +` ou `xargs -0`.

QUEM TESTA O QUÊ, E QUANDO

`for x in lista`
expande a lista **uma única vez**
↓ sobrou item?
x = próximo; executa o corpo ↺
Acrescentar arquivos ao diretório no meio de um `for f in *` **não** muda o que já foi expandido.

`for (( ; ; ))`
i=0 · i<n · i++ — contexto aritmético
↓ enquanto verdadeiro
executa o corpo ↺
A forma certa quando o limite é **variável**: `{1..$n}` não funciona — chaves expandem antes das variáveis.

`while comando`
o comando saiu com **0**?
↓ sim
executa o corpo ↺
Reavalia **a cada volta**: serve para esperar estado externo — e erro na condição vira laço infinito, não mensagem.

`until comando`
o comando saiu com **0**?
↓ **não**
executa o corpo ↺
É o `while !` escrito de forma legível: `until systemctl is-active --quiet pg; do sleep 2; done`.

O IDIOMA DE LEITURA — TRÊS DETALHES OBRIGATÓRIOS

```
# while read — linha a linha, e por campos

# IFS= preserva espacos das pontas | -r preserva a \ | < depois do done evita o subshell
$ while IFS= read -r linha || [[ -n $linha ]]; do
>     printf '[%s]\n' "$linha"
> done < dados.txt          # o || pega a ultima linha sem \n final

# com valor, o IFS divide em campos; o resto cai na ultima variavel
$ while IFS=: read -r usuario _ uid _ _ _ shell; do
>   (( uid >= 1000 && uid < 65534 )) && printf '%-12s %s\n' "$usuario" "$shell"
> done < /etc/passwd
kali         /usr/bin/bash
```

`break n` e `continue n` **saem de *n* níveis** — `break 2` abandona os dois laços aninhados e dispensa a variável `encontrado=1`. E **redirecione o laço inteiro** (`done > saida.txt`) em vez de `>>` a cada volta: o arquivo é aberto uma vez, não cinco mil.

A VARIÁVEL QUE SOME NO `done`

SHELL PAI
`total=0` continua valendo **0** depois do laço. Nenhum erro é emitido.

SUBSHELL DO PIPE
a cópia de `total` chega a 4 — e morre com o processo.

`cat arq | while read l; do (( total++ )); done` — cada estágio de um *pipe* roda no **próprio processo**. O sintoma é sempre o mesmo: valor certo *dentro* do laço, valor antigo depois do `done`.

`done < arq` redirecionamento: sem *pipe*, sem *fork*. A forma preferida quando a origem é um arquivo.

`done < <(cmd)` substituição de processo — o laço fica no pai. O **espaço entre os dois `<` é obrigatório**.

`shopt -s lastpipe` o último estágio passa a rodar no pai (exige `set +m` em script).